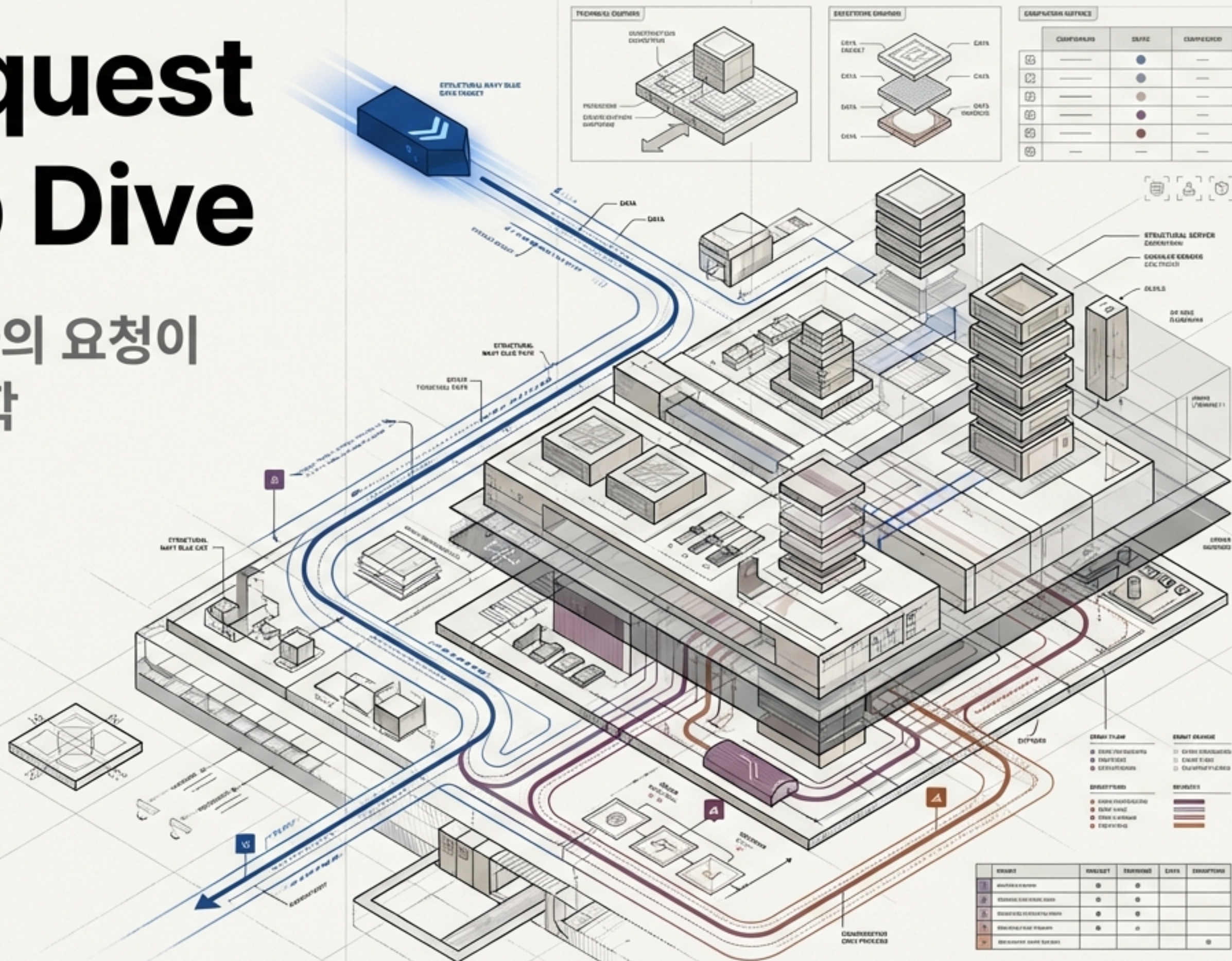
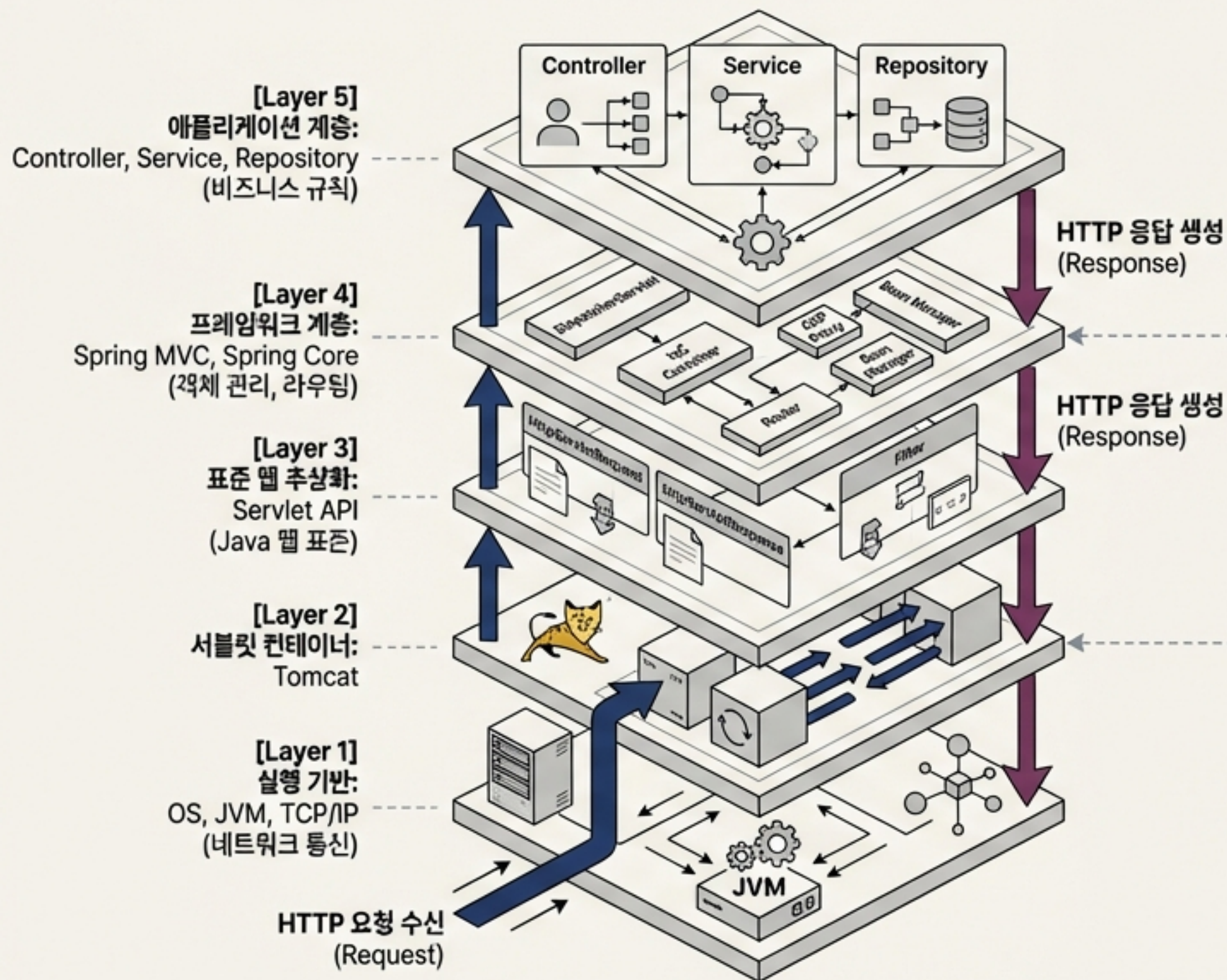


추상화 아래의 진실: 하나의 요청이 응답이 되기까지의 해부학



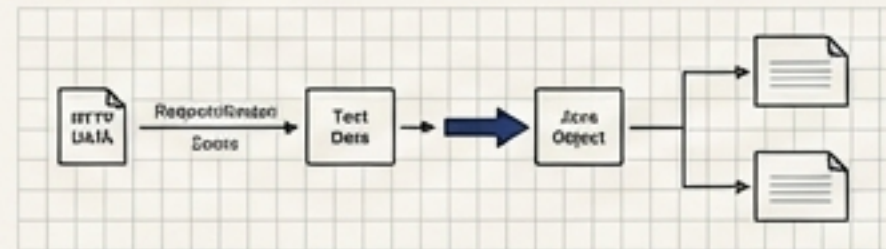
Spring은 서버의 전부가 아니다: 레이어의 분리

마법은 없다. 프레임워크는 하부 인프라 위에서 흐름을 조직할 뿐이다.



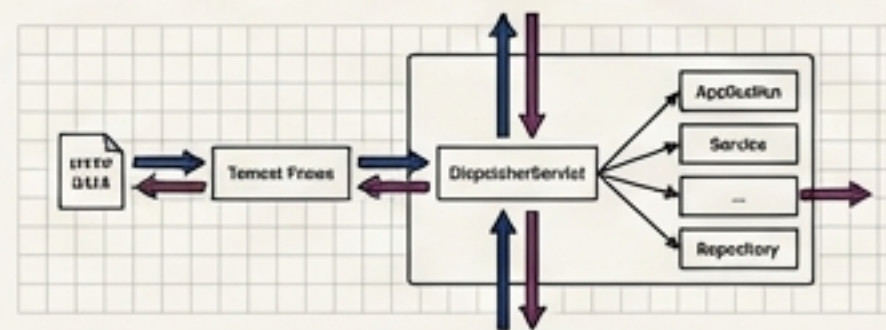
Tomcat의 역할

- HTTP 요청 수신 및 서블릿 실행 환경.
- 텍스트 메시지를 수신하여 Java 객체(HttpServletRequest)로 변환하는 독립적인 기반.



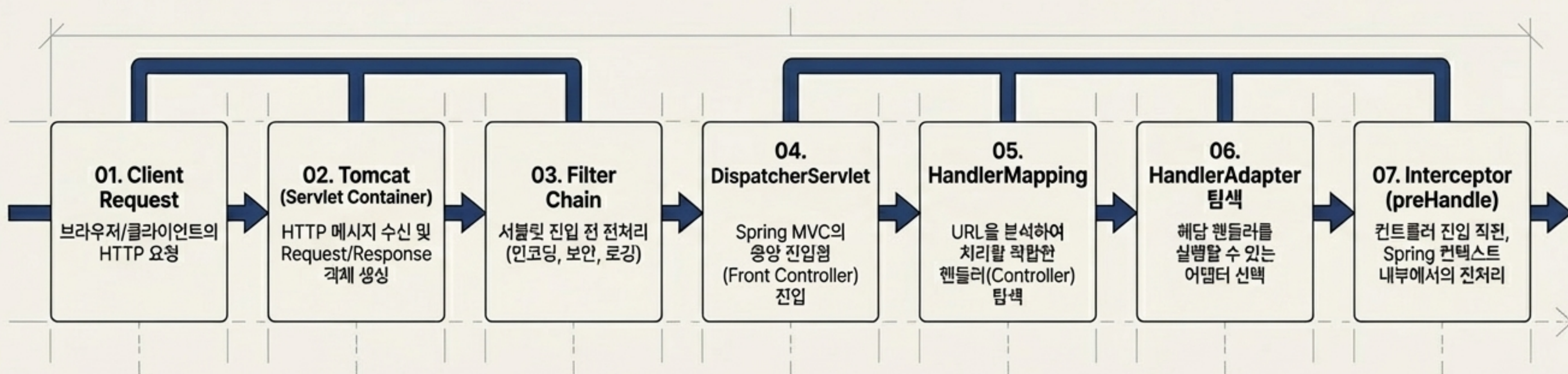
Spring의 역할

- Tomcat이 넘겨준 서블릿 요청을 받아 중앙통제실(DispatcherServlet)을 거쳐 애플리케이션 구조를 조직하고 제어하는 상위 프레임워크.



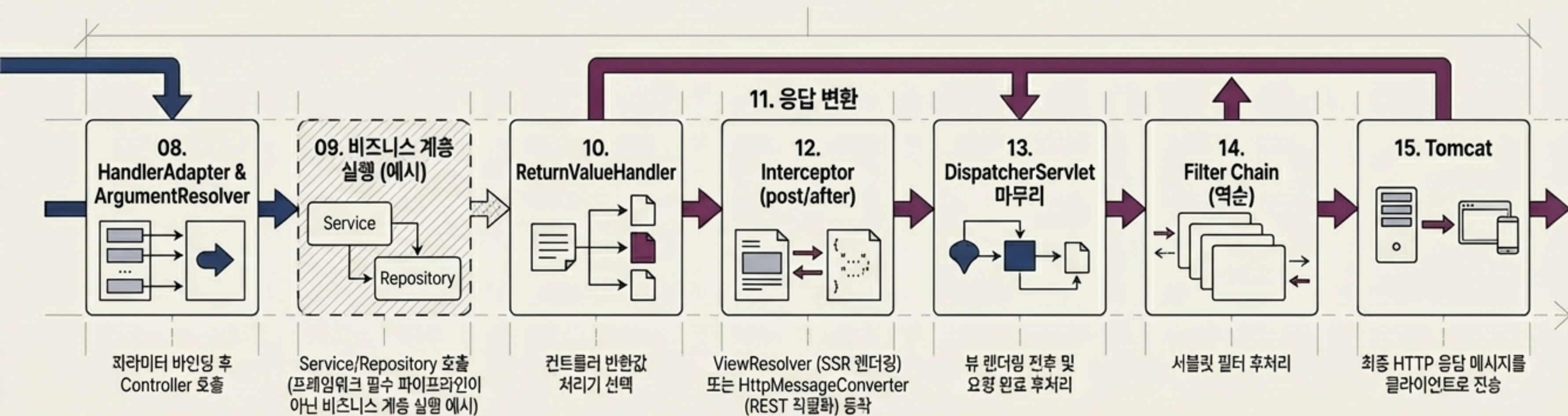
The 15-Step Pipeline (Part 1: 진입과 라우팅)

HTTP 요청이 Spring MVC의 중앙 통제실에 도달하기까지



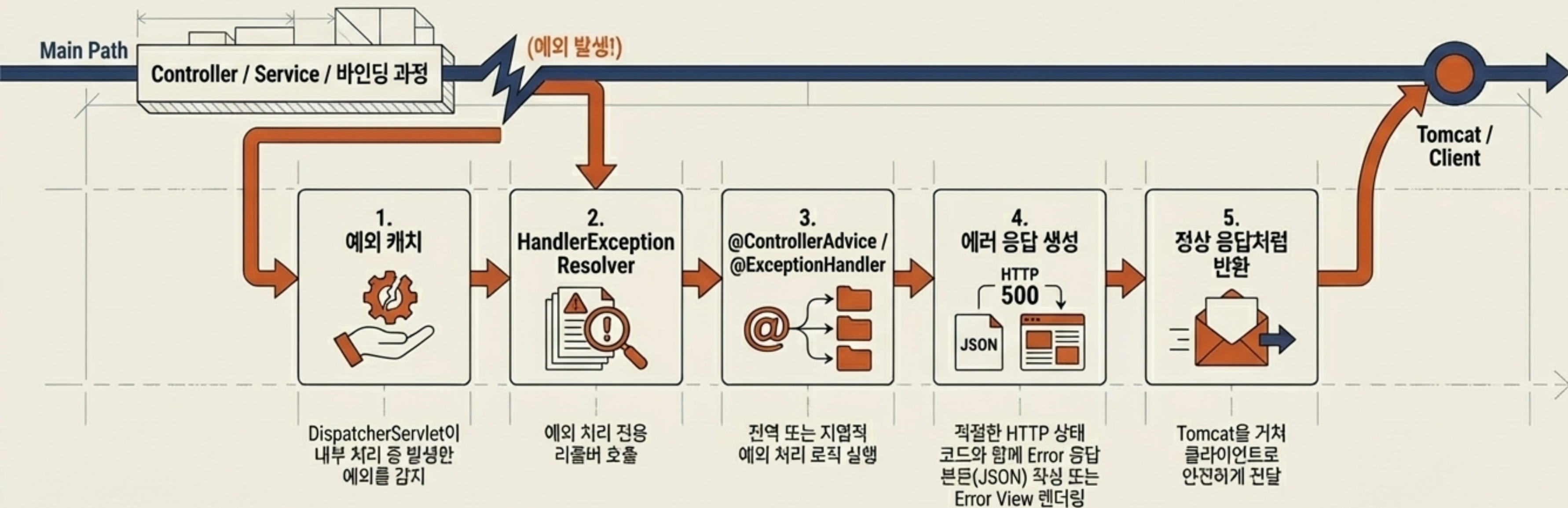
The 15-Step Pipeline (Part 2: 실행과 응답)

데이터 바인딩, 비즈니스 로직 실행, 그리고 클라이언트로의 복귀



The Shadow Pipeline: Exception Flow (예외 흐름)

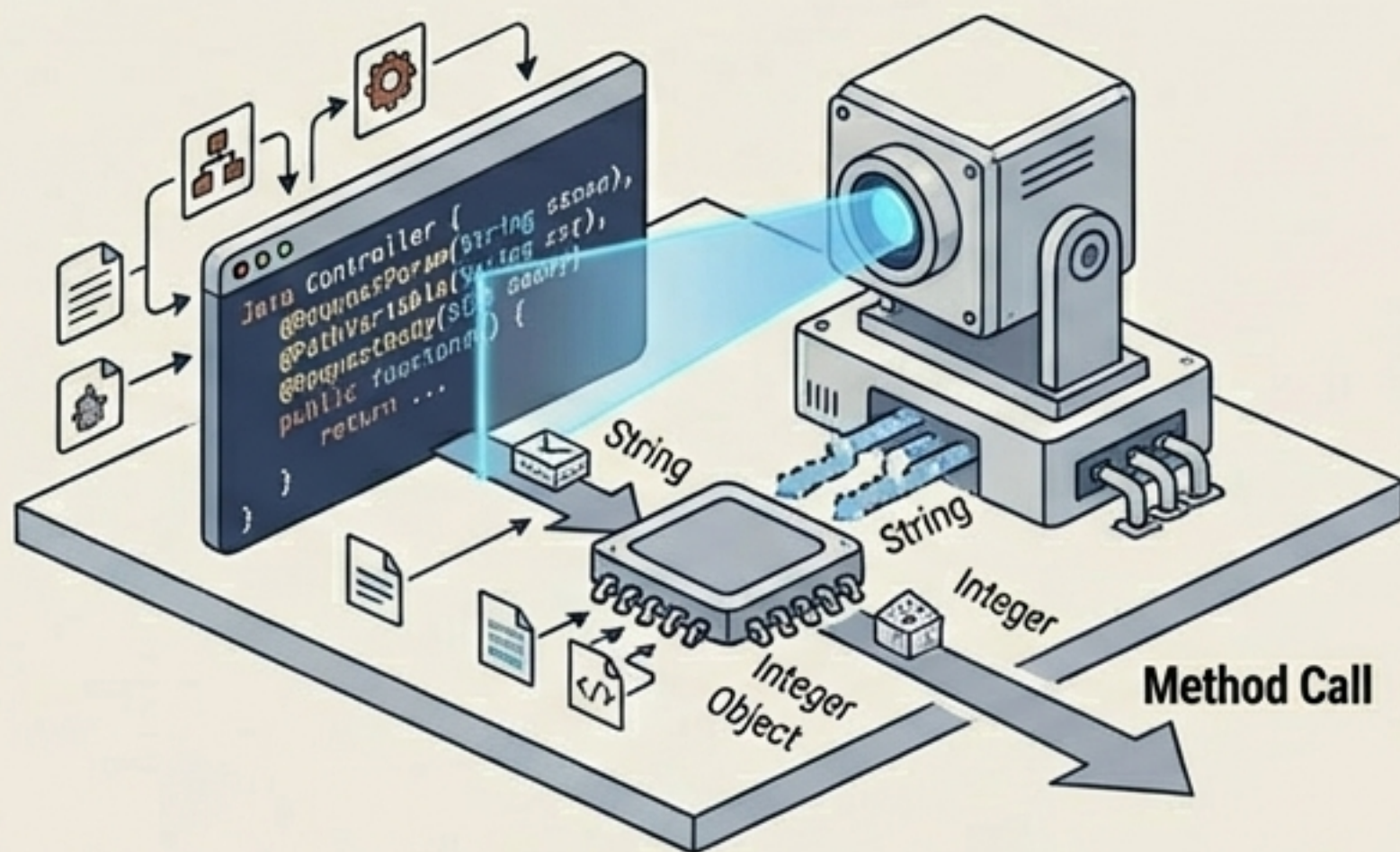
해피 패스를 벗어났을 때, 스프링은 어떻게 에러를 우아하게 잡는가



데이터의 통역사: ArgumentResolver & HttpMessageConverter

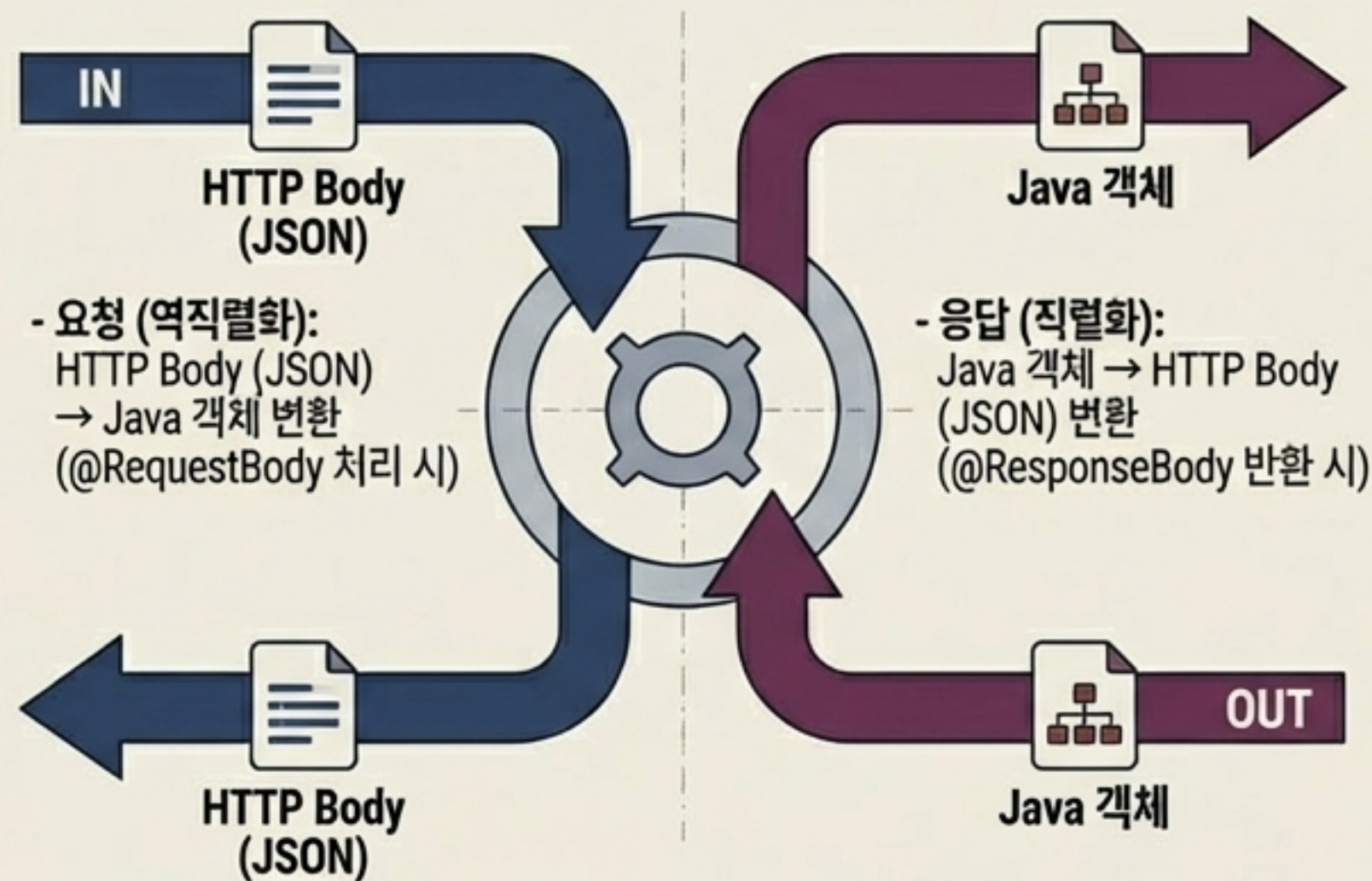
HTTP 텍스트와 Java 객체 사이의 양방향 브릿지

ArgumentResolver의 마법



- 개발자가 작성한 컨트롤러 파라미터(`@RequestParam`, `@PathVariable`, `@RequestBody` 등)를 스캔.
- 요청 데이터를 알맞은 타입으로 변환하여 메서드 호출 직전에 '주입' (파이프라인 8단계의 핵심).

HttpMessageConverter의 양방향성



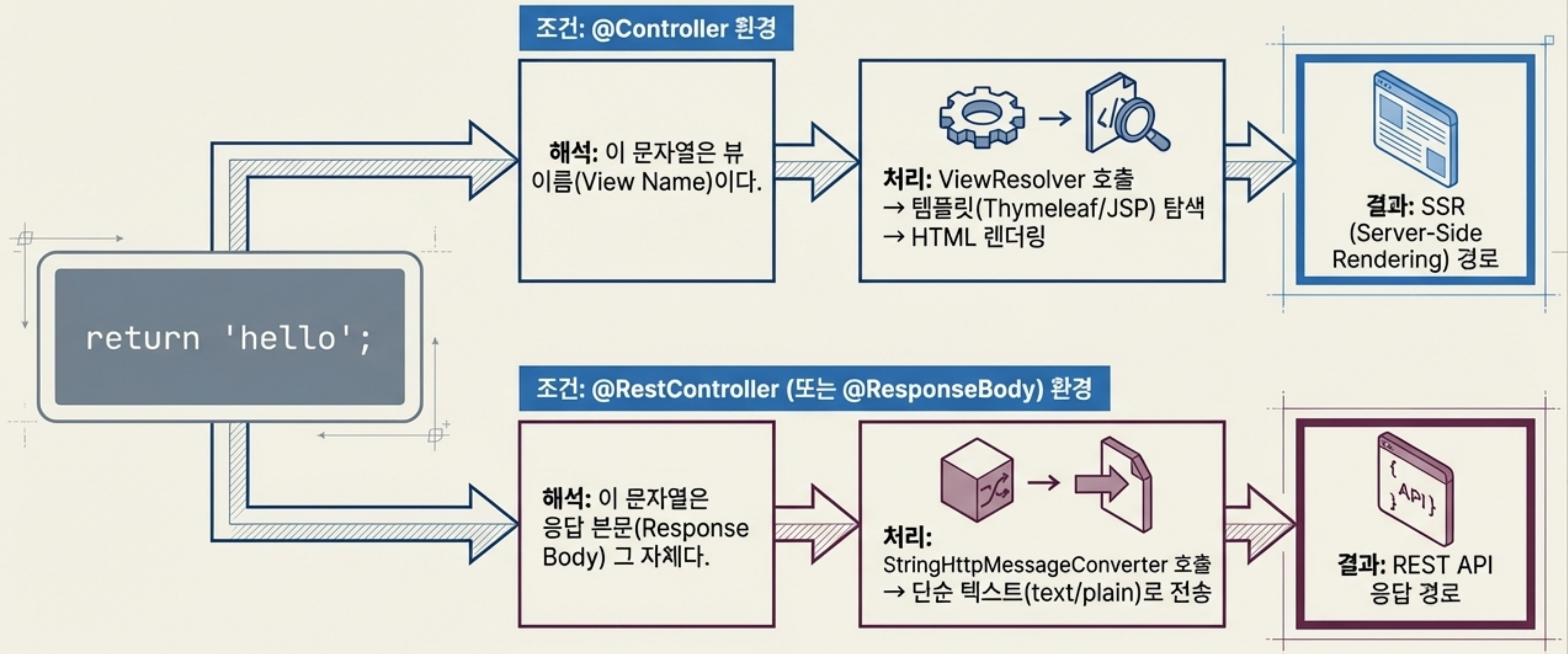
- 요청 (역직렬화):
HTTP Body (JSON)
→ Java 객체 변환
(`@RequestBody` 처리 시)

- 응답 (직렬화):
Java 객체 → HTTP Body (JSON) 변환
(`@ResponseBody` 반환 시)

- 응답 (직렬화): Java 객체 → HTTP Body (JSON) 변환
(`@ResponseBody` 반환 시)

반환값의 분기점: String이라고 무조건 SSR 이 아니다

반환값을 결정하는 것은 '타입'이 아니라 '컨텍스트'



구조적 경계: Filter vs Interceptor

요청을 가로채는 두 가지 방법, 그러나 소속된 세계가 다르다

Tomcat 영역

Spring MVC 영역

구분	Filter (필터)	Interceptor (인터셉터)
실행 위치	DispatcherServlet 이전/이후 (서블릿 영역)	DispatcherServlet 내부 (스프링 MVC 영역)
관리 주체	서블릿 컨테이너 (단, Spring Boot에선 빈 등록 가능)	Spring 컨테이너 (완전한 스프링 빈 접근 가능)
파라미터	HttpServletRequest, HttpServletResponse	Request, Response + 실행될 Handler 정보
주요 용도	전역적인 인코딩, 보안 헤더, Request 로깅, CORS	사용자 인증/인가, 컨트롤러 특정 데이터 주입

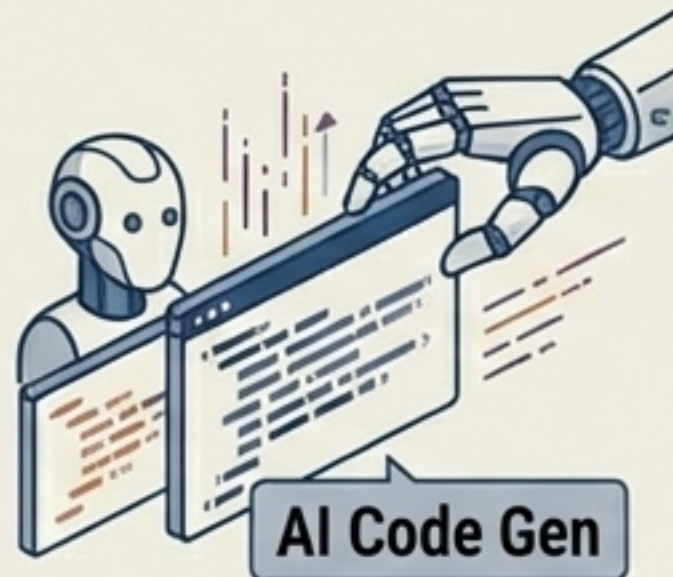
AI 시대의 시선: 코더에서 최종 검수자로

구현이 자동화될수록, 시스템을 통제하는 로우레벨 지식의 가치는 급등한다

개념 (Concept)

단순 구현 능력의 상대적 가치 하락

문법 암기, 예제 복붙, 단순 기능 추가는 AI 에이전트가 훨씬 빠르고 정확하게 수행합니다.

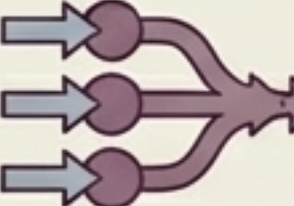
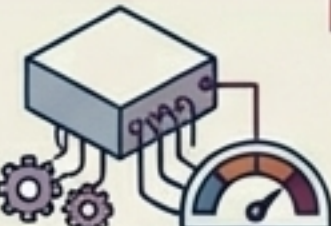


검수자(Reviewer)로서의 역할 부상

AI가 생성한 코드는 겉보기엔 그럴듯하지만, 치명적인 구조적 결함을 숨기고 있을 수 있습니다.

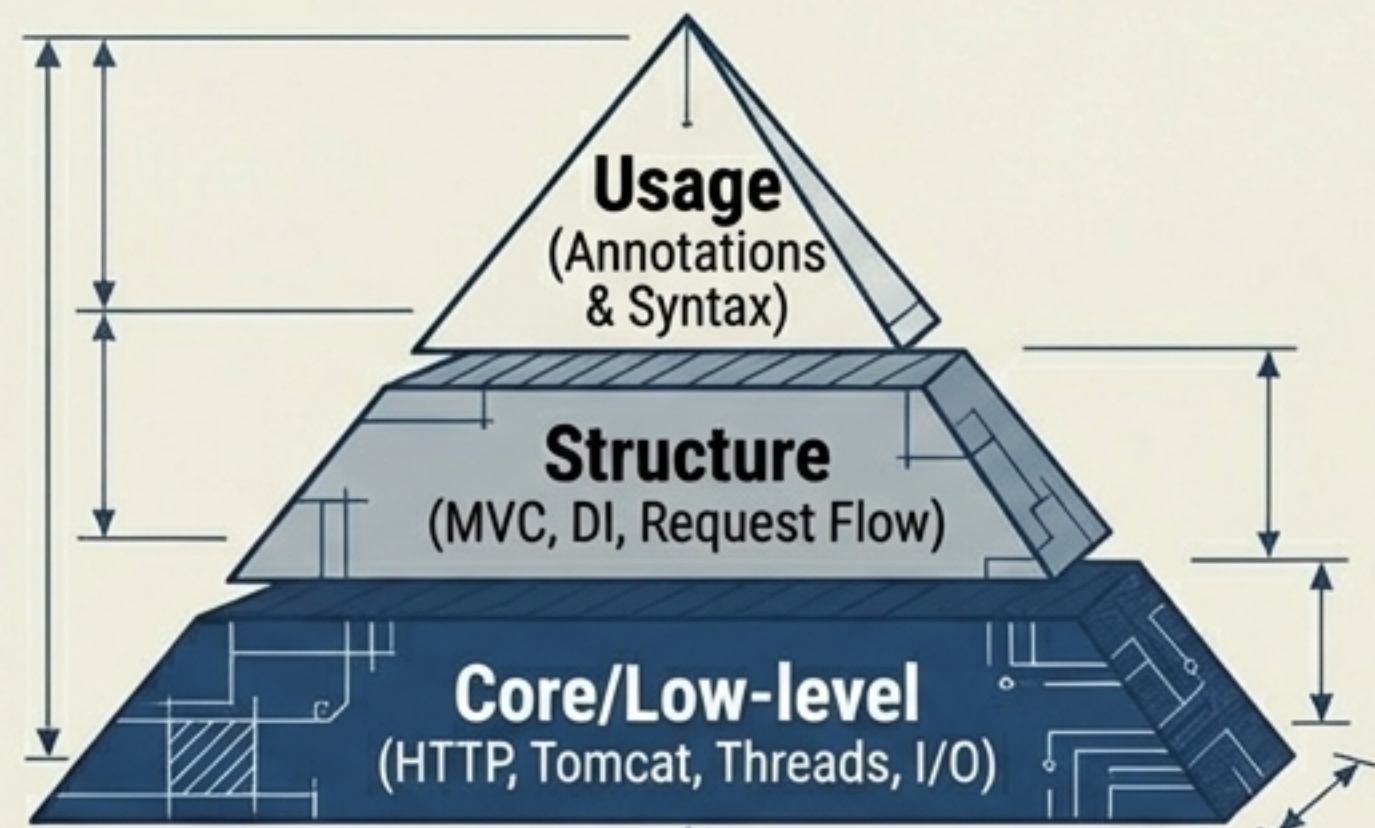


AI의 사각지대 (검수 포인트)

- ☐ **트랜잭션 경계가 올바른가?** 
- ☐ **동시성 병목이 생기지 않는가?** 
- ☐ **예외 흐름(Exception Flow)이 제대로 타는가?** 
- ☐ **추상화 아래의 실제 I/O 비용은 얼마인가?** 

추상화의 누수(Leaky Abstraction)를 통제하는 설계자

'Spring 문법'을 넘어서 '시스템 원리'를 통제하라



AI 코드를 검수하는 힘

프레임워크의 마법(추상화)이 아닌, 그 아래의 실제 동작을 아는 것에서 나옵니다.

프록시와 트랜잭션

@Transactional이 프록시 기반으로 동작함을 알아야 셀프 이노베이션(Self-invocation) 버그를 잡아냅니다.

스레드와 I/O

Tomcat의 스레드 풀과 서블릿 방식을 알아야 AI가 짠 블로킹 코드의 성능 장애를 미리 예방합니다.

결론: 프레임워크 생태계를 해부하고 통제하는 최종 구조 설계자가 되어야 합니다.